Energy-aware ventilation for the LTU A-house

D7065E, Architecture Document

Kasper Axelsson & Mohamd Amar Chms Aldden

Contents

1. Summary **1**

2. Use case and building context **2**

3. Requirements **2**

4. Architecture **3**

5. Interfaces and data contracts **7**

6. Simulating the sensor values (the physical model) **8**

7. Autonomous services and data pipeline **8**

8. Behaviour **10**

9. Deployment and component view **12**

10. Test plan **12**

11. Evaluation and results **13**

12. Dashboard **14**

13. Risks and critical reflection **15**

1. Summary

The system ventilates and heats 50 rooms of the A-house in BuildSim according to who is in them. A controller reads CO_2, temperature and a people count from sensor devices, forecasts each room's occupancy from a learned time-of-day profile, and commands a damper and a radiator valve per room. One principle runs through the design: **the controller proposes, the building disposes.** The controller knows the building only through what sensor devices report, and every safety limit is enforced in the actuator, which keeps working when the controller, the broker or a sensor does not. Over a simulated week the system holds CO_2 under 1000 ppm in every occupied minute for 57 % less energy than ventilation on a time clock.

Project title	Energy-aware ventilation and climate control for the LTU A-house
Team members	Kasper Axelsson, Mohamd Amar Chms Aldden
Use case	Demand-controlled ventilation with occupancy prediction
Repository	https://github.com/MisterD0ctor/d7065e-project (design notes, contracts, test plan and review log in `docs/`)
Version / date	2026-10-11

Changes since the proposal. (1) *Occupancy is measured, not estimated.* The proposal inverted the CO_2 mass balance to estimate head-count; for lack of time each room has a people counter instead, the fallback the proposal named. The mass balance survives as a plausibility check on that counter (Section 7.1). (2) *The data path was rebuilt on 10 October.* We had read "the controller must not see the truth" as "the truth must not enter BuildSim" and built a truth API

beside BuildSim. That is not the lab's architecture; the system now follows the quickstart, with BuildSim in the middle (Section 4.4, D-12). All results in this report were measured after that change. (3) *The baseline is a time clock*, not design flow around the clock, which no building runs at night and which had inflated the saving from 57 % to 75 %.

2. Use case and building context

BuildSim holds the A-house: the floor plan, equipment with sensor values and actuator states, the people, and a 3D viewer. It stores current state only; it simulates no physics, keeps no history and has no clock. The course's occupancy simulator fills it with staff, students and night guards following a seeded daily plan. We chose 50 rooms on level 0: 3 common (fika) rooms, 20 lecture rooms and 27 small rooms, 24 of them in use as offices.

Ventilating by a schedule wastes energy on empty rooms and still under-ventilates full ones. Ventilating by measured CO_2 reacts only after the air has degraded, and a room's air responds slowly (time constant 12 to 60 minutes). The objective is to keep CO_2 below 1000 ppm and temperature within 20–24 °C while anyone is in a room, for as little fan and heating energy as possible, and to stay safe when a sensor, the controller or the network fails.

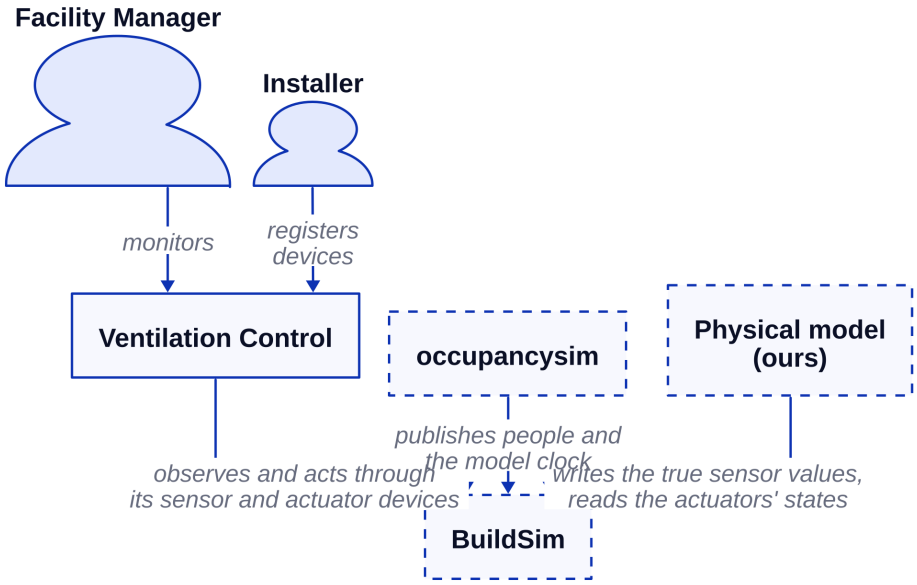
3. Requirements

ID	Requirement	Verified by
FR-1	A physical model simulates CO_2 and temperature per room, first order, driven by true occupancy and the airflow and setpoint the actuators have reached; steps $\leq$ 1 model minute	T-01, T-41
FR-2	Each room has a CO_2, a temperature and an occupancy sensor device; each reports once per model minute with noise: CO_2 $\pm(30\,\text{ppm} + 3\,\%)$, temperature $\pm 0.3\,°C$, count ± 1	T-03, T-41, T-42
FR-3	The controller sets airflow and heating setpoint for every room once per minute	T-05
FR-4	It forecasts each room's occupancy one lead time ahead from a model trained offline	T-06, T-35
FR-5	The actuator validates every command (range, unit, TTL, duplicate) and travels at a fixed rate	T-07, T-08
FR-6	Safety in the damper, independent of the controller: $CO_2 > 1100$ ppm gives maximum flow; never below $0.35\,\text{l/s}\,\text{m}^2$ in occupied hours, 0.10 otherwise	T-09, T-10, T-11
FR-7	A stale or stuck sensor is detected and the room falls back: design flow without CO_2, 21 °C without temperature, forecast and CO_2 without the count	T-12, T-13, T-36–38
FR-8	Every reading, true value, command and decision is stored with model time and run id	T-14, T-43
FR-9	Four policies selectable by a flag; nothing else differs between runs	T-15, T-32
FR-10	A dashboard shows each room and the building, now and over time, with alerts	T-16, T-39
FR-11	Devices do nothing until an installer has registered them; BuildSim's equipment is restored after a restart	T-17, T-28–31
NFR-1	$CO_2 \leq 1000$ ppm for $\geq 95\,\%$ of occupied room-minutes	T-18
NFR-2	20–24 °C for $\geq 95\,\%$ of occupied room-minutes	T-19
NFR-3	Energy $\geq 30\,\%$ below the scheduled baseline	T-20
NFR-4	A command shows in the damper's state within one model minute (95th percentile)	T-21
NFR-5	50 rooms: every sensor's reading arrives every model minute	T-22, T-42
NFR-6	Any one of our processes can be lost without an unsafe state; the loop resumes within 30 s	T-23, T-24, T-40
NFR-7	Same seed and date give identical occupancy; every number traces to a run id	T-25

ID	Requirement	Verified by
NFR-8	One baseline over five simulated weekdays in ≤ 2.5 h	T-26

4. Architecture

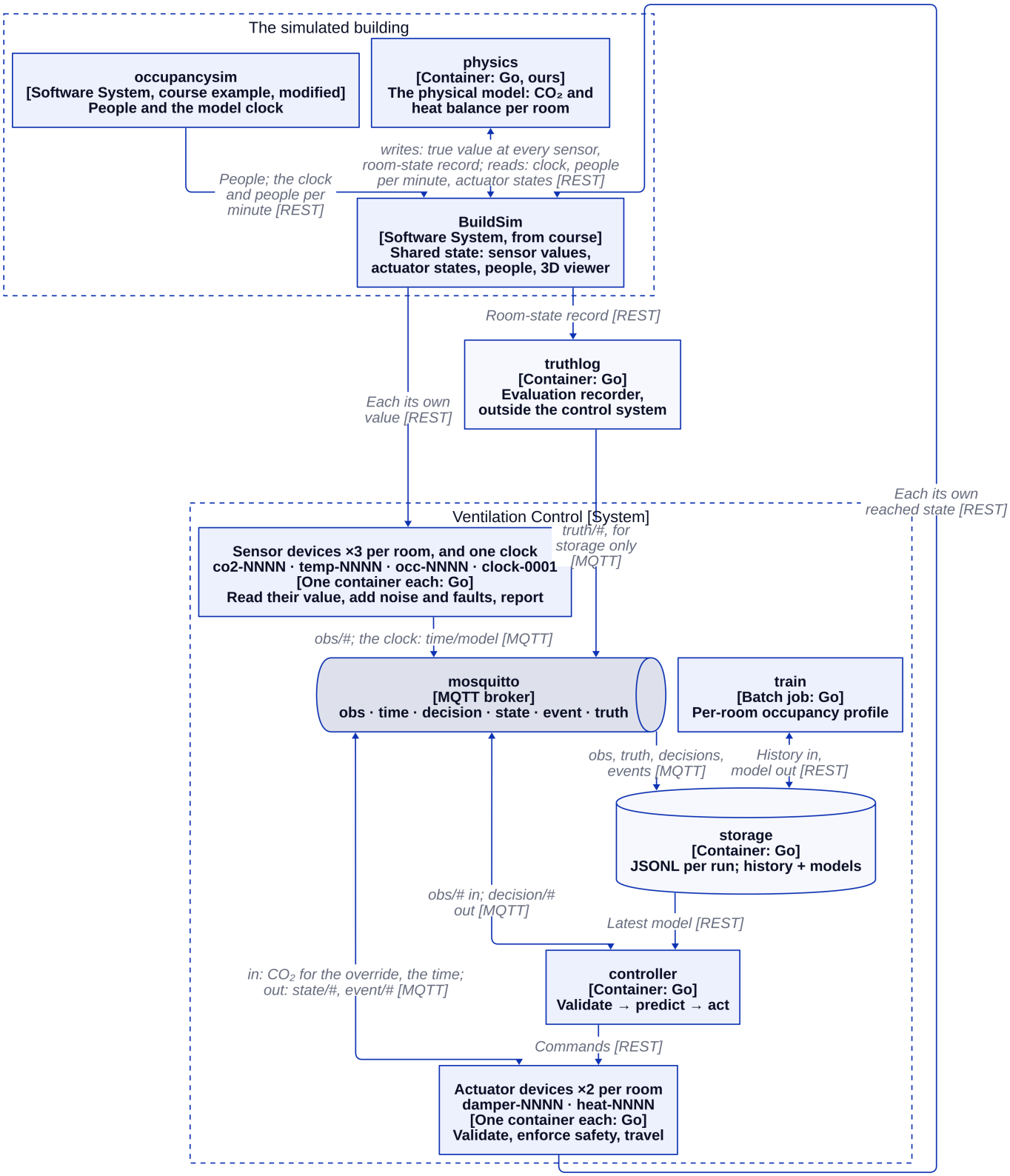
4.1 Context diagram (C4 Level 1)



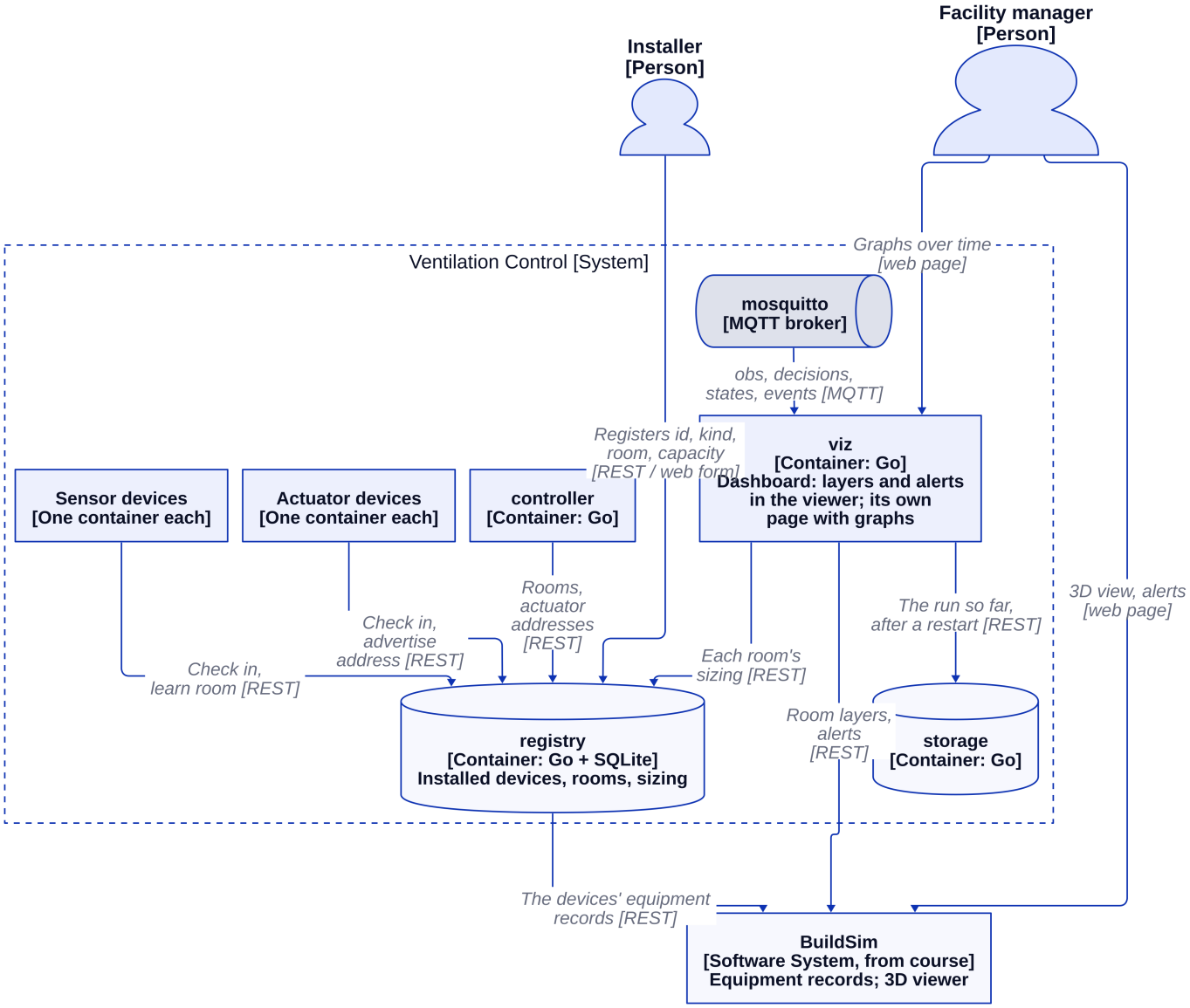
The simulated building is three things: BuildSim (course), the occupancy simulator (course example, modified) and our physical model. They are outside the control system on purpose: together they stand in for a real building. *Ventilation Control* touches the building only through its sensor and actuator devices. A facility manager watches the dashboard; an installer registers devices.

4.2 Container diagram (C4 Level 2)

The container level is drawn as two views, because with every link in one picture the labels are too small to read on a page. The first is the control loop and the data path. The second is commissioning and the dashboard; boxes with the same name are the same containers. A two-way arrow names both directions.



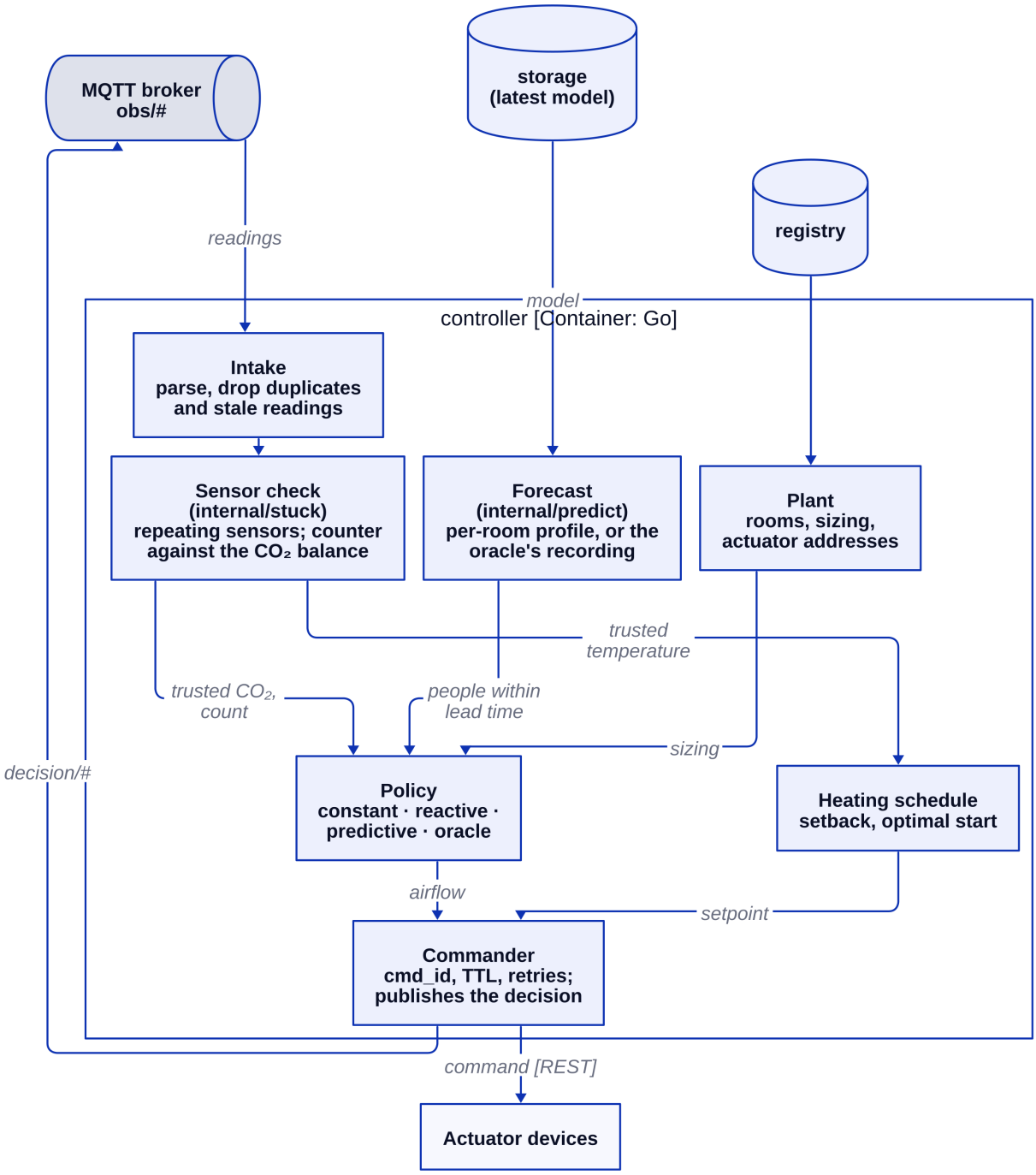
View 1: the control loop and the data path.



View 2: commissioning and the dashboard.

Every sensor and actuator is its own container (251 devices), so one dead device affects one room. The occupancy simulator and the physical model talk to BuildSim and to nothing else. A sensor device reads the true value at its place in BuildSim, adds its own measurement error and publishes the reading on MQTT. The controller subscribes, decides, and commands each room's actuators over REST; an actuator applies the command within its safety limits and writes the state it has reached to BuildSim, where the physical model reads it back. That closes the loop. Storage records every message; the dashboard (viz) draws from the same messages. The registry holds what the installer entered: device id, kind, room and the room's design capacity. The truth recorder (truthlog) is outside the control system: it copies the physical model's state to a broker topic only storage may read, for the evaluation.

4.3 Component diagram (C4 Level 3): the controller



Validation, forecast and policy sit in one process because they are one critical path with no reason to be restarted separately. Readings pass a sensor check before the policy sees them. The forecast is a model fetched from storage and kept in memory, so storage may be down. The commander gives each command an id and a time to live and retries with the same id, which the actuator treats as one command.

4.4 Design decisions and alternatives

Decision	Alternatives considered	Why this one
D-12 BuildSim in the middle: the two simulators talk only to BuildSim; the controller reaches it only through devices	A truth API on the physical model beside BuildSim, and the controller polling BuildSim as a fallback (built first)	It is the lab's architecture, and one representation of the building instead of two. The controller is kept from the truth by having no BuildSim address. Cost: the truth is visible in the viewer, and there is no control without the broker
D-1 Model time; physics steps whole model minutes from a per-minute occupancy record	Each service on its own wall-clock times factor; sampling BuildSim's occupancy	Reproducible: two runs of a day are identical. Sampling gave 99.6 % equal samples and made the oracle baseline unsound
D-2 One container per sensor; noise and faults in the device	One gateway per kind; noise in the physical model	A fault is a property of one device and can be injected on one. Cost: about 150 containers, generated
D-3 MQTT for readings, REST for commands	Everyone polling BuildSim	Four consumers of each reading (controller, damper, storage, dashboard). A command needs an answer, so it is a request
D-4 A storage service owning JSONL per run	Files shared between processes; a database server	No shared files; replayable. A history query reads the whole run, too slow for live graphs, so viz keeps its own recent copy
D-5 Predictor inside the controller; training as a batch job	Predictor as a service	No network hop on the control path; training off it
D-6 Safety in the actuator: range, rate, TTL, flow floors, CO_2 override	A check inside the controller	Survives a wrong or dead controller (T-10)
D-7 Four policies; the oracle replays a recorded run's true occupancy	Oracle reading the simulator's plans	Shows how much a perfect forecast could gain. Needs identical days (D-1)
D-8 Dashboard: layers and alerts in the BuildSim viewer, and a page of graphs served by viz	Grafana over storage; the viewer alone	The viewer shows now; ventilation questions are about time. One HTML file, no dependencies
D-10 A registry (SQLite) that devices check in with	Device lists in configuration; registering directly in BuildSim	How a building is commissioned; survives BuildSim restarting blank (T-17)
D-11 Stuck sensors: a run of identical readings for CO_2 and temperature; the counter against two bounds from the CO_2 balance	"Counter unchanged while CO_2 moves"; estimating head-count from CO_2 and airflow	The first blames every healthy counter (CO_2 lags the count); the second needs the true airflow. The bounds hold for any airflow
Room set: 50 rooms	Whole building (838 rooms)	260 containers is what one laptop runs, after two kernel settings

5. Interfaces and data contracts

Rooms are keyed `<level>/<room>`; times are model time, RFC 3339. Full contracts with failure behaviour are in `docs/interfaces.md` (IF-1 to IF-12).

Interface	Owner	Content
BuildSim sensor `sim-clock-minutes`	occupancy simulator	Clock, and people per room for the last ten model minutes
BuildSim `PUT /api/sensors/{id}/value`	physics	True value at every sensor, once per minute; the minute itself, last, at the clock device
MQTT `obs/<level>/<room>/<kind>`	each sensor	The reading, QoS 1; consumers drop exact repeats of `(sensor_id, seq)`
MQTT `time/model`	clock device	The building's minute, on change and every real second
REST `POST /rooms/{l}/{r}/command`	actuator	`202` target, `200` duplicate, `409` expired, `422` invalid

Interface	Owner	Content
BuildSim `PUT /api/actuators/{id}/state`; MQTT `state/...`	each actuator	The state it has reached
MQTT `decision/...`, `event/...`	controller; actuators	Every decision with its mode and reason; overrides and expiries
MQTT `truth/...`	truthlog	True room state per minute; the broker lets only storage read it
REST storage, registry		History and models; devices, rooms, check-in

```
obs/level0/A1519/co2  {"run_id":"eval-predictive-v4","sensor_id":"co2-0041","room":"level0/A1519","kind":"co2",
                       "value":439,"unit":"ppm","model_time":"2026-09-21T08:21:00Z","seq":1789978860}
command               {"cmd_id":"a99e...","value":308.0,"unit":"l/s","issued_at":"2026-09-21T10:15:00Z",
                       "ttl_s":300,"reason":"predictive: CO2 612 ppm, 25 people within 12 min"}
```

6. Simulating the sensor values (the physical model)

Per room, with n people, airflow q, volume V, floor area A:

$$V\frac{dC}{dt} = n\,G + q\,(C_{\text{out}} - C), \qquad c_A A\,\frac{dT}{dt} = UA\,(T_{\text{out}} - T) + \rho c_p\,q\,(T_{\text{sup}} - T) + 100\,\text{W}\cdot n + P_{\text{rad}}$$

with $G = 0.005\,\text{l/s}$ of CO_2 per person, $C_{\text{out}} = 420\,\text{ppm}$, ceiling 2.7 m, $T_{\text{out}} = 8\,°\text{C}$, supply air 18 °C, and a radiator of $60\,\text{W/m}^2$ that is fully on 0.5 K below its setpoint. Both equations are solved exactly over each model minute. The loop closes because q and the setpoint are *the states the actuators report in BuildSim*, not the commands. The energy proxy is fan power (1.5 kW per m^3/s), heating the supply air, and radiator power.

Ventilation is sized by the Swedish workplace rule: $q_{\text{design}} = 7\,\text{l/s}$ per person of design capacity $+ 0.35\,\text{l/s}\,\text{m}^2$; the maximum is 1.5 times that. A full room at design flow settles near 1000 ppm, which is why the maximum is higher.

Occupancy comes from the course's simulator: a seeded plan per day with staff in offices, lectures in four slots in rooms drawn at random, breaks at 09:45 and 14:30, and night guards. We changed it in three logged ways: fixed 5-second steps, a record of people per room at every whole minute, and publishing that record and its clock into BuildSim. A sensor reading is the true value plus Gaussian noise (a ±1 miscount in 10 % of readings for the counter), and injected faults: stuck, dropout, drift.

7. Autonomous services and data pipeline

7.1 The autonomous service

Policies. *Constant*: design flow on weekdays 07–18, the minimum otherwise. *Reactive*: flow rises linearly from the minimum at 600 ppm to the maximum at 1000 ppm. *Predictive*: the larger of the reactive flow and the workplace-rule flow for the people expected, where "expected" is the larger of the count now and the forecast within the room's lead time (its time constant, 10 to 30 minutes). *Oracle*: the same with the true future occupancy of a recorded run. Heating is the same under all four: 21 °C on weekdays 07–18, 18 °C otherwise, starting early enough to be warm at 07:00.

The learned part is a per-room profile: for each weekday quarter-hour, the 80th percentile across training days of that slot's occupancy. Within a day a slot's value is the 75th percentile of its readings, not the maximum: taking the maximum let the counter's occasional false "1" make the model expect someone in 86 % of all slots, nights included, and cost 25 % more energy until we found it. Without a model the policy runs as reactive and says so.

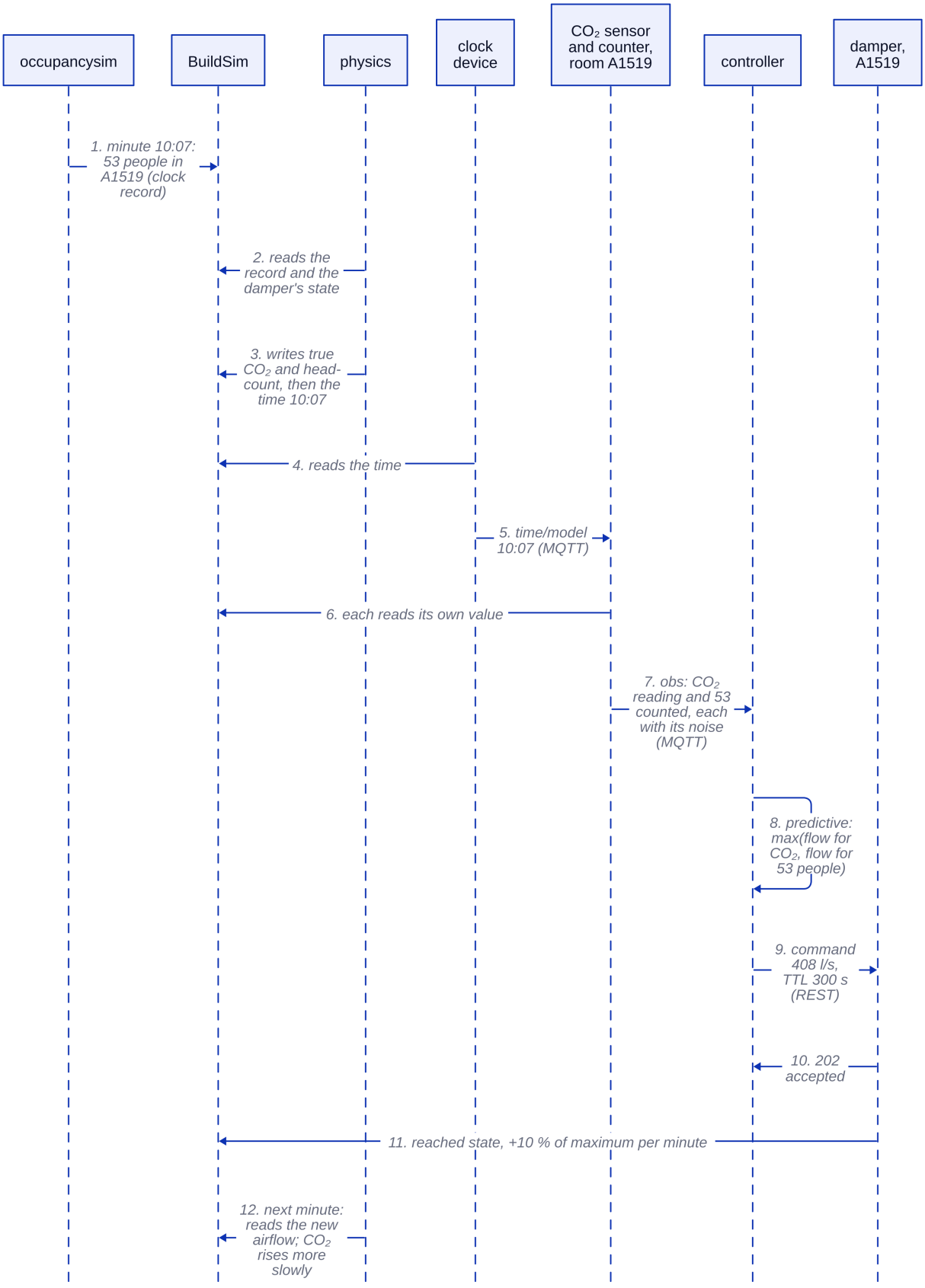
Stuck sensors keep reporting, so silence does not reveal them. A CO_2 or temperature sensor is noisy, so 5 (or 12) identical readings in a row cannot be chance: those are the shortest runs a healthy 50-room building produces less than once in 100 days. A counter legitimately reads 0 for hours, so it is checked against the CO_2 balance instead: with x the CO_2 above outdoor, x cannot rise faster than nG/V, nor stay below $(nG/q_{\max})(1 - e^{-t\,q_{\max}/V})$, whatever the airflow. A counter that has not moved and breaks either bound is distrusted.

7.2 The data pipeline

Sensors publish on MQTT. Storage appends every reading, truth record, decision, event and actuator state to one JSONL file per stream and run, and serves history, exports and models over HTTP; nothing is deleted. The trainer is a batch job that reads a training run from storage and stores a model, which the controller picks up. The controller keeps only the latest readings; the dashboard keeps a week of minutes in memory and refills from storage after a restart.

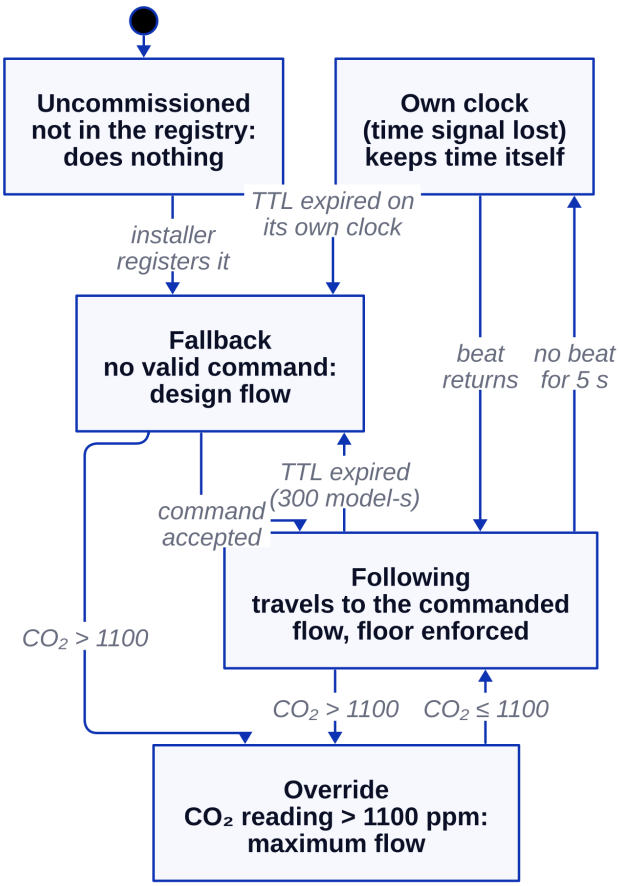
8. Behaviour

8.1 Key scenario: a lecture room fills



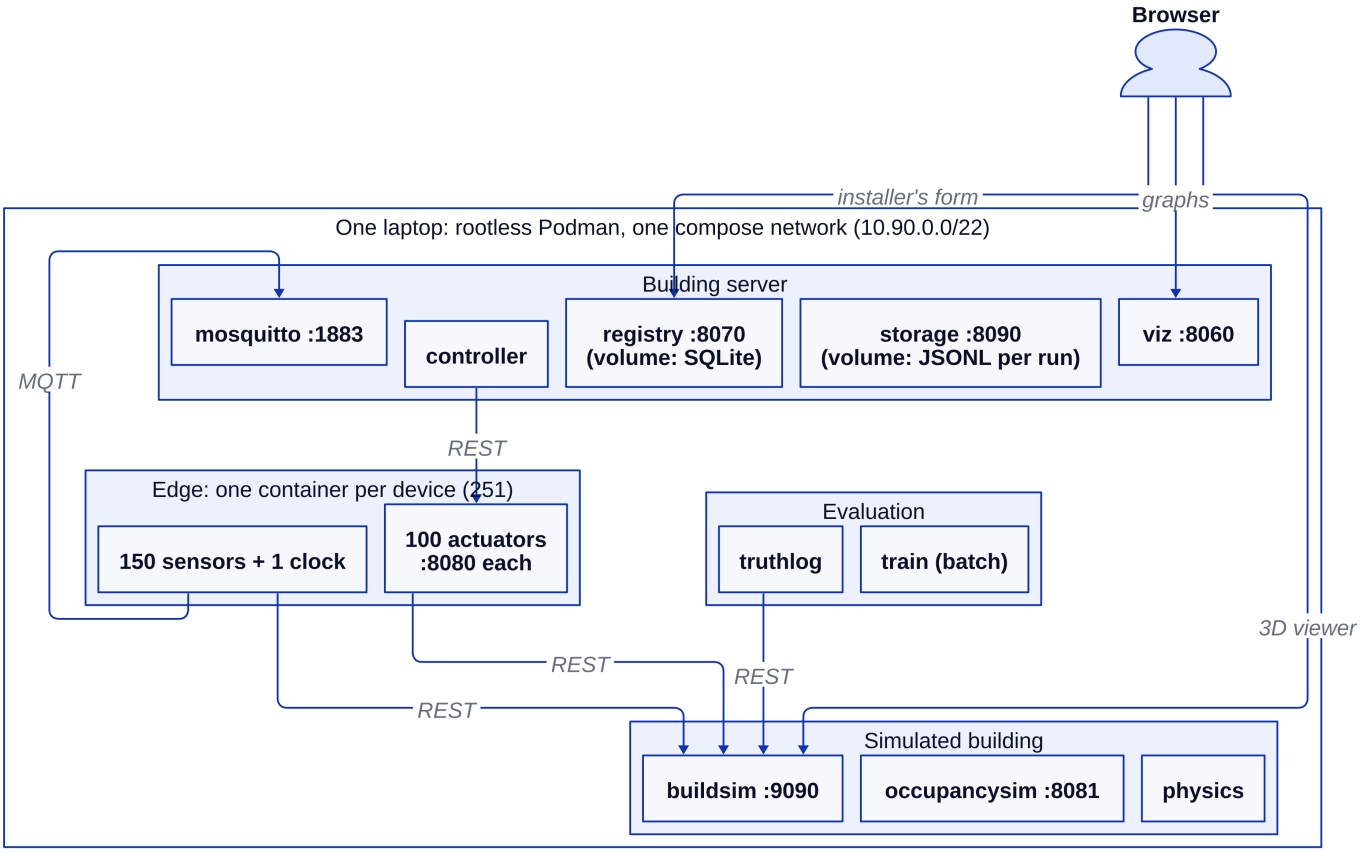
The order of steps 3 to 6 is the point: the physical model writes the minute to the clock device *after* that minute's values, and sensors sample when the clock shows a new minute, so each reading belongs to its minute. At factor 120 every one of 150 sensors delivered 120 of 120 readings (T-42).

8.2 Process lifecycle: a damper



A damper without a valid command is at design flow. Commands expire after 300 model seconds, so a silent controller, sensor or broker returns the room to design flow without anyone deciding it. If the time signal itself stops, the damper keeps time at the last known speed so that the expiry still happens.

9. Deployment and component view



Everything runs as containers on one laptop under one compose network, generated from the installation list. In a building, each device container would be a device, and the grouping in the figure is where the rest would run: sensing, validation of commands and the safety limits at the edge, where they must survive the network; prediction, storage and the dashboard on a building server; training offline.

10. Test plan

43 tests; the full table with every result is `docs/test-plan.md`. Unit tests (`go test`, 20 packages) cover the model, noise, policies, actuator, detector and each piece of the data path. The live tests are three scripts, each its own run id.

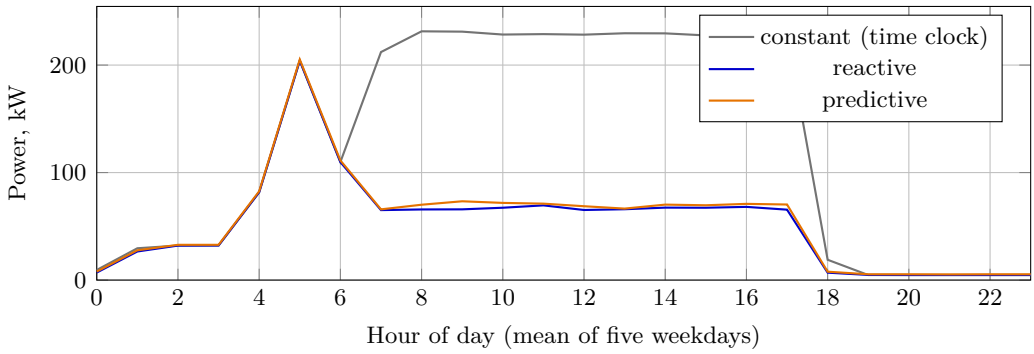
ID	Test	Req.	Result
T-09/10	Sensor drifts above 1100 ppm with the controller killed	FR-6	Damper at its maximum, 315.6 l/s, with no controller running
T-12	Counter frozen at 53 during a lecture	FR-7	Not blamed while the room was full; found 14 minutes after it emptied; flow 408 to 371/s
T-13	CO_2 sensor stopped	FR-7	Command expired after 5 minutes; damper to design flow; cleared on return
T-36	CO_2 sensor frozen; temperature sensor frozen	FR-7	Found after 5 and 12 readings. 12 people in the room; true CO_2 peaked at 571 ppm
T-17	BuildSim restarted blank	FR-11	253 pieces of equipment back in 7.8 s; readings in 9.1 s
T-23	Each of nine services killed in turn	NFR-6	Loop closed in all rooms within 2 s of restart. **No container restarted by itself** (Podman)
T-24, T-40	Broker stopped; clock device stopped	NFR-6	All 100 actuators expired their commands on their own clocks; design flow; recovered alone

ID	Test	Req.	Result
T-21	Command to damper movement	NFR-4	93 % within one model minute; **95th percentile 2: missed**
T-22 (stress)	Simulation speed doubled until readings are lost	NFR-5	100 % at factor 120, 79 % at 240, 25 % at 960
T-25	Same week under four policies and two data paths	NFR-7	360 500 truth records, every head-count equal
T-27	Controller's login reads the truth topic	scope	Receives nothing; has no BuildSim address
T-38	Stuck-sensor detector over fault-free runs	FR-7	No false alarm in 1.44 million readings of each kind
T-43	What storage holds of a run	FR-8	Every reading from the first minute; found a bug that discarded a run's readings

11. Evaluation and results

A training week (14–18 Sep 2026) under the reactive policy, then the held-out week 21–25 Sep once per policy, at 120 times real time. The four runs saw the same people in every room at every minute, so differences are the policies'.

	Constant	Reactive	Predictive	Oracle
$CO_2 \leq 1000$ ppm, occupied (NFR-1)	94.5 %	100.0 %	100.0 %	100.0 %
in lecture rooms	65.2 %	100.0 %	100.0 %	100.0 %
Occupied minutes above 1000 ppm	745	4	0	0
Peak CO_2, ppm	1167	1028	963	963
20–24 °C, occupied (NFR-2)	100.0 %	100.0 %	100.0 %	100.0 %
Energy, kWh per week	15 231	6 284	6 519	6 372
Saving against constant (NFR-3)		59 %	57 %	58 %



Reading. A time clock costs two and a half times as much and still fails: a full lecture room settles around 1000 ppm at design flow, and anyone in an office outside 07–18 sits at the night minimum. Reactive control already does almost everything: four minutes over the limit in a week, at the lowest energy. A perfect forecast (oracle) removes those four minutes for 1.4 % more energy; our predictor gets the same air for 3.7 % more. That small gap between reactive and oracle is itself the result: the dampers reach full flow in minutes and lecture rooms catch up fast, so there is little for anticipation to win. The peak at 05:00 in the figure is the same under every policy: radiators bringing the rooms back from the night setback.

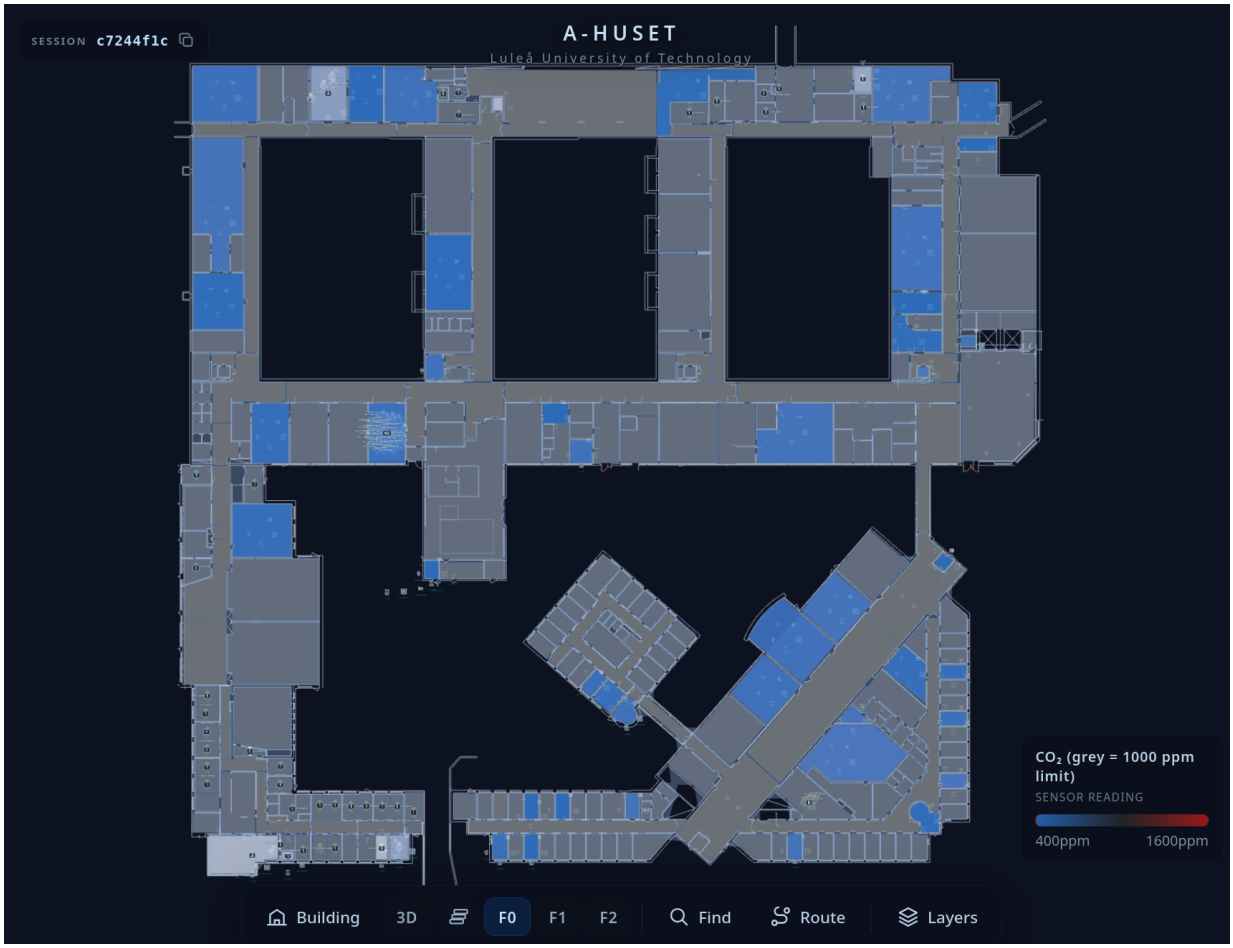
Where the numbers are weak. The forecast foresaw 87 % of arrivals in common rooms, with false alarms in 23 % of quiet moments, and *none* in lecture rooms: the simulator puts each lecture in a random room, so a room has no pattern to learn. It is one seed and one week. The building is a model we wrote, so the result says as much about its time constants as about the controllers. NFR-4 is missed at the 95th percentile (T-21), and above factor 120 readings are lost (T-22).

A check on the rework. The same week had been measured on the first data path. After

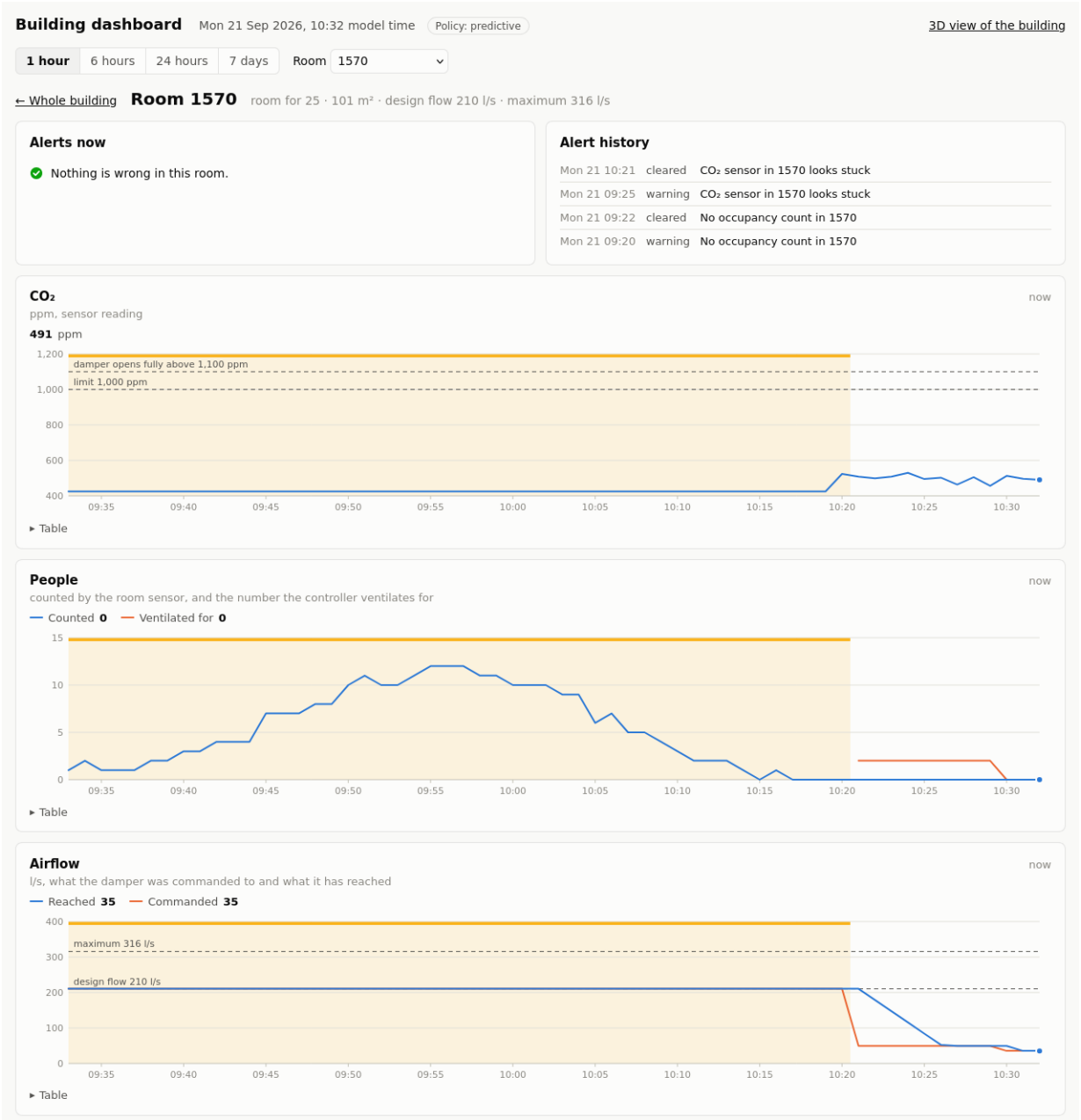
rebuilding it, reactive, predictive and oracle came out at 6 284, 6 519 and 6 372 kWh against 6 282, 6 521 and 6 370: the architecture changed, the behaviour did not.

12. Dashboard

The dashboard is one process, viz, with two faces. In the BuildSim 3D viewer it publishes five room layers (CO_2, people counted, people expected, airflow as a share of the room's maximum, temperature) and the current alerts. On its own page (port 8060) it shows the building and each room over time, up to a week back.



The first figure is the viewer on level 0 with the CO_2 layer: each room we instrument is coloured by its sensor's latest reading, on a scale from 400 to 1600 ppm with grey at the 1000 ppm limit, so blue is air well under the limit.



The second figure is a room whose CO_2 sensor was frozen at 09:20: the flat reading, the alert period shaded, the damper held at design flow while up to 12 people came and went, and its rate-limited return when the sensor was repaired. Everything shown arrived on MQTT from a sensor, the controller or an actuator: the dashboard shows the building as the control system knows it, never the truth.

13. Risks and critical reflection

We built the wrong data path first. For ten days the physical model had its own API and the controller could read BuildSim. We noticed two days before the deadline and rebuilt it. It cost a night of reruns and bought the lab's architecture, exact sampling, and one finding we would otherwise have missed: storage had been discarding readings whenever time moved backwards, which the new start-up order exposed.

The prediction earns little here. It spends 3.7 % more energy to remove four minutes above the limit. With slower dampers or tighter limits it would matter more; a lecture timetable as an

input would let it foresee lectures, which it cannot now.

The truth is visible in BuildSim. Clicking a device in the viewer shows the true value. The controller is kept from it by configuration and by the broker's access list, not by authentication, which BuildSim does not have.

Known limits. No control without the broker (the actuators then take safe, wasteful defaults). A counter stuck on a small number is not found: below about a quarter of a room's capacity its CO_2 is inside the sensor noise. A drifting sensor is not detected. A killed container is not restarted automatically under Podman. Devices do not authenticate to the registry. The heating's warm-up rate is assumed, not learned.

On working with an AI assistant. Most code was written with one. The review log (`docs/review-log.md`) records each piece of its advice that was wrong and how we caught it: the data path above, a baseline that ventilated all night, a training rule that turned sensor noise into people. Each was caught by measuring, not by reading the code.

With more time: a second seed and a longer evaluation; the lecture timetable; learning the warm-up rate; a per-run blank BuildSim; network isolation of BuildSim from the controller.